



Conflicting Appointments (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity
- Similar Problems

Problem Statement

Given an array of intervals representing 'N' appointments, find out if a person can **attend all the appointments**.

Example 1:

Appointments: [[1,4], [2,5], [7,9]]

Output: false

Explanation: Since [1,4] and [2,5] overlap, a person cannot attend both of these appointments.

Example 2:

Appointments: [[6,7], [2,4], [8,12]]

Output: true

Explanation: None of the appointments overlap, therefore a person can attend all of them.

Example 3:

Appointments: [[4,5], [2,3], [3,6]]

Output: false

Explanation: Since [4,5] and [3,6] overlap, a person cannot attend both of these appointments.

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

Tf

```
1 import java.util.*;
2
3 class Interval {
```



```

4   int start;
5   int end;
6
7   public Interval(int start, int end) {
8       this.start = start;
9       this.end = end;
10  }
11  };
12
13  class ConflictingAppointments {
14
15      public static boolean canAttendAllAppointments(Interval[] intervals) {
16          // TODO: Write your code here
17          return true;
18      }
19
20      public static void main(String[] args) {
21          Interval[] intervals = { new Interval(1, 4), new Interval(2, 5), new Interval(7, 9) };
22          boolean result = ConflictingAppointments.canAttendAllAppointments(intervals);
23          System.out.println("Can attend all appointments: " + result);
24
25          Interval[] intervals1 = { new Interval(6, 7), new Interval(2, 4), new Interval(8, 12) };
26          result = ConflictingAppointments.canAttendAllAppointments(intervals1);
27          System.out.println("Can attend all appointments: " + result);
28
29          Interval[] intervals2 = { new Interval(4, 5), new Interval(2, 3), new Interval(3, 6) };
30          result = ConflictingAppointments.canAttendAllAppointments(intervals2);
31          System.out.println("Can attend all appointments: " + result);
32      }
33  }
34

```

Run

Save

Reset



Solution

The problem follows the [Merge Intervals](#) pattern. We can sort all the intervals by start time and then check if any two intervals overlap. A person will not be able to attend all appointments if any two appointments overlap.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class Interval {
4      int start;
5      int end;
6
7      public Interval(int start, int end) {
8          this.start = start;
9          this.end = end;
10     }
11 };
12
13 class ConflictingAppointments {
14
15     public static boolean canAttendAllAppointments(Interval[] intervals) {
16         // sort the intervals by start time
17         Arrays.sort(intervals, (a, b) -> Integer.compare(a.start, b.start));

```



```

18
19 // find any overlapping appointment
20 for (int i = 1; i < intervals.length; i++) {
21     if (intervals[i].start < intervals[i - 1].end) {
22         // please note the comparison above, it is "<" and not "<="
23         // while merging we needed "<=" comparison, as we will be merging the two
24         // intervals having condition "intervals[i].start == intervals[i - 1].end" but
25         // such intervals don't represent conflicting appointments as one starts right
26         // after the other
27         return false;
28     }
29 }
30 return true;
31 }
32
33 public static void main(String[] args) {
34     Interval[] intervals = { new Interval(1, 4), new Interval(2, 5), new Interval(7, 9) };
35     boolean result = ConflictingAppointments.canAttendAllAppointments(intervals);
36     System.out.println("Can attend all appointments: " + result);
37
38     Interval[] intervals1 = { new Interval(6, 7), new Interval(2, 4), new Interval(8, 12) };
39     result = ConflictingAppointments.canAttendAllAppointments(intervals1);
40     System.out.println("Can attend all appointments: " + result);
41
42     Interval[] intervals2 = { new Interval(4, 5), new Interval(2, 3), new Interval(3, 6) };
43     result = ConflictingAppointments.canAttendAllAppointments(intervals2);
44     System.out.println("Can attend all appointments: " + result);
45 }
46 }
47

```

Run

Save

Reset

⌘

Time complexity

The time complexity of the above algorithm is $O(N * \log N)$, where 'N' is the total number of appointments. Though we are iterating the intervals only once, our algorithm will take $O(N * \log N)$ since we need to sort them in the beginning.

Space complexity

The space complexity of the above algorithm will be $O(N)$, which we need for sorting. For Java, `Arrays.sort()` uses [Timsort](#), which needs $O(N)$ space.

Similar Problems

Problem 1: Given a list of appointments, find all the conflicting appointments.

Example:

Appointments: [[4,5], [2,3], [3,6], [5,7], [7,8]]

Output:

[4,5] and [3,6] conflict.

[3,6] and [5,7] conflict.

Tt



[← Back](#)

 Completed

[Next →](#)

Intervals Intersection (medium)

Problem Challenge 1

